SAVITRIBAI PHULE PUNE UNIVERSITY
B.E. Artificial Intelligence and Data Science
Semester VIII (2020 Course)

QUESTION BANK & NOTES

Subject Code	Subject Name	Credits
417530	Computational Intelligence (CI)	03
417531	Distributed Computing (DC)	03

Examination: In-Sem: 30 Marks | End-Sem: 70 Marks

No.	Unit	CI Topic	DC Topic
I	Unit 1	Introduction to Computational Intelligence	Introduction to Distributed Computing
II	Unit 2	Fuzzy Logic	Distributed Data Management & Storage
III	Unit 3	Evolutionary Computing	Distributed Computing Algorithms
IV	Unit 4	Genetic Algorithm	Distributed Machine Learning and AI
V	Unit 5	Computational Intelligence and NLP	Big Data Processing in Distributed Systems
VI	Unit 6	Artificial Immune Systems	Distributed Systems Security and Privacy

417530: Computational Intelligence

Unit I: Introduction to Computational Intelligence

2-Mark Questions

Q1. [2M] What is Computational Intelligence (CI)?

Computational Intelligence is a set of nature-inspired computational methodologies and approaches that address complex real-world problems where mathematical or traditional modelling is not feasible. It includes fuzzy logic, neural networks, and evolutionary computing.

Q2. [2M] What are the main paradigms of Computational Intelligence?

The three main paradigms are: 1) Fuzzy Logic - deals with uncertainty and imprecision, 2) Artificial Neural Networks - inspired by biological brain, 3) Evolutionary Computation - inspired by natural evolution. These three are often called the CI triumvirate.

Q3. [2M] Differentiate between Artificial Intelligence (AI) and Computational Intelligence (CI).

- AI: Rule-based, symbolic reasoning, logic-driven, deterministic
- CI: Nature-inspired, handles uncertainty, learns from data, adaptive
- AI uses crisp logic; CI uses soft computing and approximate reasoning
- CI is a subset of AI focused on bio-inspired algorithms

Q4. [2M] What are the applications of Computational Intelligence?

- Pattern recognition and image processing
- Medical diagnosis and healthcare
- Control systems and robotics
- Financial forecasting and prediction
- Natural language processing
- Optimization problems

Q5. [2M] What are the grand challenges of Computational Intelligence?

Grand challenges include: scalability to handle big data, interpretability of models, real-time processing, combining multiple CI techniques effectively, handling dynamic and non-stationary environments, and bridging the gap between CI and human cognition.

5-Mark Questions

Q6. [5M] Explain the synergies of Computational Intelligence techniques.

CI techniques work together to solve complex problems:

- Neuro-Fuzzy Systems: Combines neural networks with fuzzy logic. Neural networks learn parameters of fuzzy systems automatically.
- Evolutionary Neural Networks: Genetic algorithms optimize neural network architecture and weights.
- Fuzzy Evolutionary Systems: Fuzzy logic guides evolutionary search, handles uncertainty in fitness evaluation.
- Neuro-Evolutionary Systems: Evolution creates neural network topology automatically.

These synergies allow each technique to compensate for weaknesses of others, creating more powerful hybrid systems.

Q7. [5M] What are the different approaches to Computational Intelligence? Explain each.

- Fuzzy Systems: Handle imprecision and uncertainty using fuzzy sets and logic. Good for linguistic reasoning.

• Neural Networks: Learn from examples, identify patterns. Inspired by biological brain neurons.

• Evolutionary Computation: Population-based search inspired by Darwinian evolution. Good for optimization.

• Swarm Intelligence: Based on collective behavior of decentralized systems (ants, bees, birds).

• Probabilistic Methods: Handle uncertainty using probability distributions and Bayesian reasoning.

10-Mark Questions

Q8. [10M] Explain the paradigms of Computational Intelligence with their characteristics and applications.

1. FUZZY LOGIC:

• Based on fuzzy set theory by Lotfi Zadeh (1965)

• Handles linguistic uncertainty and vagueness

• Uses membership functions (0 to 1) instead of crisp 0/1

• Applications: washing machines, air conditioners, control systems

2. NEURAL NETWORKS:

• Inspired by biological neural structure of brain

• Learn from training data through backpropagation

• Good for pattern recognition, classification, regression

• Applications: image recognition, speech recognition, NLP

3. EVOLUTIONARY COMPUTATION:

• Based on Darwinian natural selection and genetics

• Population-based optimization using selection, crossover, mutation

• Good for complex optimization and search problems

• Applications: scheduling, routing, design optimization

4. SWARM INTELLIGENCE:

• Inspired by collective behavior of social insects

• Ant Colony Optimization (ACO), Particle Swarm Optimization (PSO)

• Good for distributed optimization problems

• Applications: network routing, logistics, robot swarms

Unit II: Fuzzy Logic

2-Mark Questions

Q1. [2M] What is a Fuzzy Set? How is it different from a crisp set?

A Fuzzy Set is a set where each element has a degree of membership between 0 and 1. Unlike crisp sets where an element either belongs (1) or does not belong (0), fuzzy sets allow partial membership.

• Crisp set: student is pass (1) or fail (0)

• Fuzzy set: student has 0.7 degree of passing

Q2. [2M] What is a Membership Function?

A Membership Function (MF) maps each element of a universe of discourse to a membership value between 0 and 1. It defines how each element belongs to a fuzzy set. Common types: Triangular, Trapezoidal, Gaussian, Sigmoid.

Q3. [2M] What are Linguistic Variables and Hedges?

Linguistic Variables are variables whose values are words or sentences in natural language rather than numbers. Example: 'Temperature' with values 'cold', 'warm', 'hot'. Hedges are modifiers that adjust the

meaning: 'very', 'somewhat', 'extremely', 'not'. Example: 'very hot', 'somewhat cold'.

Q4. [2M] What is Fuzzification and Defuzzification?

Fuzzification: The process of converting crisp input values into fuzzy membership values using membership functions. Example: temperature 35 degrees becomes 0.7 in 'hot' set.

Defuzzification: Converting fuzzy output back to crisp values. Methods: Centroid, MOM (Mean of Maximum), COG (Center of Gravity).

Q5. [2M] What are the types of Fuzzy Logic Controllers?

- Mamdani FLC: Uses fuzzy rules with fuzzy consequents. More interpretable.
- Takagi-Sugeno FLC: Consequents are mathematical functions. More computationally efficient.
- Tsukamoto FLC: Each rule has monotonic membership function.

5-Mark Questions

Q6. [5M] Explain Fuzzy Set Operations with formulas and examples.

Given: A = {a:0.5, b:0.7, c:0.2}, B = {b:0.6, c:0.8, d:0.4}

- UNION: degree(A∪B) = max(A(x), B(x)). Example: b = max(0.7,0.6) = 0.7
- INTERSECTION: degree(A∩B) = min(A(x), B(x)). Example: b = min(0.7,0.6) = 0.6
- COMPLEMENT: degree(A') = 1 - A(x). Example: a = 1-0.5 = 0.5
- DIFFERENCE: degree(A-B) = min(A(x), 1-B(x)). Example: b = min(0.7, 0.4) = 0.4

Q7. [5M] Explain the Fuzzy Inference System (FIS) with steps.

- Step 1 - Fuzzification: Convert crisp inputs to fuzzy values using MFs
- Step 2 - Rule Evaluation: Apply fuzzy rules (IF-THEN) to fuzzy inputs
- Step 3 - Aggregation: Combine outputs of all rules into single fuzzy set
- Step 4 - Defuzzification: Convert fuzzy output to crisp value

Example Rule: IF temperature IS hot AND humidity IS high THEN fan_speed IS fast

10-Mark Questions

Q8. [10M] Explain Fuzzy Logic with Cartesian Product and Max-Min Composition.

CARTESIAN PRODUCT: Creates a fuzzy relation from two fuzzy sets.

- R = A x B where R(x,y) = min(A(x), B(y)) for all x in A and y in B
- Creates all possible pairs and assigns min of membership values

MAX-MIN COMPOSITION:

- Given R (relation A to B) and S (relation B to C)
- Composition T = R o S gives relation from A to C
- T(x,z) = max over all y of [min(R(x,y), S(y,z))]
- Why MIN? - Chain is as strong as its weakest link
- Why MAX? - Find the best possible path between two points

EXAMPLE: A={a:0.5,b:0.7,c:0.2}, B={b:0.6,c:0.8,d:0.4}, C={x:0.3,y:0.9,z:0.5}

- R = AxB: (a,b)=0.5, (a,c)=0.5, (a,d)=0.4, (b,b)=0.6, (b,c)=0.7, ...
- S = BxC: (b,x)=0.3, (b,y)=0.6, (c,x)=0.3, (c,y)=0.8, ...
- T(a,y) = max(min(R(a,b),S(b,y)), min(R(a,c),S(c,y))) = max(0.5,0.5) = 0.5

Unit III: Evolutionary Computing

2-Mark Questions

Q1. [2M] What is Evolutionary Computing?

Evolutionary Computing (EC) is a family of algorithms inspired by biological evolution (Darwinian natural selection). It uses population-based search with mechanisms like selection, crossover, and mutation to solve optimization and search problems.

Q2. [2M] What are the terminologies of Evolutionary Computing?

- Individual/Chromosome: A single candidate solution
- Population: Collection of all individuals in current generation
- Fitness Function: Measures how good a solution is
- Selection: Choosing individuals for reproduction
- Crossover: Combining two parents to create offspring
- Mutation: Random change in an individual
- Generation: One iteration of the evolutionary process

Q3. [2M] What is Ant Colony Optimization (ACO)?

ACO is a swarm intelligence algorithm inspired by the foraging behavior of ants. Ants deposit pheromones on paths to food. Shorter paths get more pheromone. Algorithm uses artificial pheromones to find optimal paths/solutions. Used for TSP, routing, scheduling problems.

Q4. [2M] Differentiate Evolutionary Computing and Classical Optimization.

- EC: Population-based, stochastic, no gradient needed, global search
- Classical: Single solution, deterministic, needs gradient, local search
- EC: Good for complex non-linear, multi-modal problems
- Classical: Good for smooth, differentiable, well-defined problems

5-Mark Questions

Q5. [5M] Explain the types of Evolutionary Algorithms.

- 1. Genetic Algorithm (GA): Uses binary/real encoding, crossover, mutation. Good for combinatorial optimization.
- 2. Evolution Strategies (ES): Real-valued parameters, self-adaptive mutation rates. Used in continuous optimization.
- 3. Evolutionary Programming (EP): Focuses on evolving behavior, no crossover, only mutation.
- 4. Genetic Programming (GP): Evolves computer programs represented as trees. Used for automatic programming.

All share common steps: Initialize population → Evaluate fitness → Select → Reproduce → Repeat

Q6. [5M] Explain Multi-objective Optimization in Evolutionary Computing.

Multi-objective optimization involves optimizing two or more conflicting objectives simultaneously.

- Example: Minimize cost AND maximize quality - improving one worsens the other
- Pareto Optimality: A solution is Pareto optimal if no objective can be improved without worsening another
- Pareto Front: Set of all Pareto optimal solutions
- Methods: NSGA-II, MOEA/D, SPEA2
- Applications: Engineering design, portfolio optimization, scheduling

10-Mark Questions

Q7. [10M] Explain Swarm Intelligence with ACO and its applications in detail.

SWARM INTELLIGENCE: Collective behavior of decentralized, self-organized systems.

ANT COLONY OPTIMIZATION (ACO):

- Inspired by foraging behavior of ants finding shortest path to food

- Ants deposit pheromone on paths; shorter paths accumulate more pheromone
- Other ants prefer paths with higher pheromone concentration
- Pheromone evaporates over time preventing convergence to suboptimal paths

ACO ALGORITHM STEPS:

- 1. Initialize: Place ants at random starting positions, initialize pheromone trails
- 2. Construction: Each ant builds solution using pheromone and heuristic information
- 3. Pheromone Update: Deposit pheromone on path proportional to solution quality
- 4. Evaporation: Reduce all pheromone trails by evaporation factor
- 5. Repeat until stopping criterion

PHEROMONE UPDATE FORMULA:

- tau(i,j) = (1-rho)*tau(i,j) + delta_tau(i,j)
- rho = evaporation rate, delta_tau = pheromone deposited by ant

APPLICATIONS:

- Traveling Salesman Problem (TSP)
- Vehicle routing and logistics
- Network routing in telecommunications
- Job scheduling in manufacturing
- Protein folding problem in bioinformatics

Unit IV: Genetic Algorithm

2-Mark Questions

Q1. [2M] Define the basic terminologies in Genetic Algorithm.

- Gene: Smallest unit of chromosome, represents one parameter
- Chromosome: Complete set of genes, represents one solution
- Individual: Single organism with its chromosome
- Population: Group of individuals in current generation
- Fitness Function: Measures how good the solution is
- Allele: Specific value of a gene
- Genotype: Genetic representation, Phenotype: Expressed solution

Q2. [2M] What are the representations used in Genetic Algorithm?

- Binary Representation: Chromosome as string of 0s and 1s. Simple and most common.
- Floating-Point Representation: Real-valued genes. Better for continuous problems.
- Integer Representation: For discrete problems.
- Permutation Representation: For ordering problems like TSP.
- Tree Representation: Used in Genetic Programming.

Q3. [2M] What is the stopping condition in GA?

- Maximum number of generations reached
- Fitness value reaches a satisfactory threshold
- No significant improvement in last N generations
- Maximum time limit reached
- Population has converged (all individuals are similar)

Q4. [2M] What is the difference between Genotype and Phenotype?

Genotype is the genetic representation of a solution (the chromosome/encoding). Phenotype is the actual expressed solution in the problem domain. Example: Genotype = binary string '10110', Phenotype = the actual value or solution it represents (22 in decimal).

5-Mark Questions

Q5. [5M] Explain the operators in Genetic Algorithm with examples.

1. SELECTION: Choose parents based on fitness. Types:
   - Roulette Wheel: Probability proportional to fitness
   - Tournament: Pick best from random subset
   - Rank Selection: Select based on rank not fitness value
2. CROSSOVER (Recombination): Combine two parents to create child. Types:
   - Single-point: Split at one point and swap
   - Two-point: Swap segment between two points
   - Uniform: Each gene independently chosen from either parent
3. MUTATION: Random change in chromosome.
   - Bit Flip: Flip 0 to 1 or 1 to 0 (binary)
   - Gaussian: Add small random value (real-valued)
   - Swap: Swap two gene positions (permutation)

Q6. [5M] Explain GA with Traveling Salesman Problem (TSP) example.

TSP: Find shortest route visiting all cities exactly once and returning to start.

- Representation: Permutation encoding - [1,3,2,4,5] means visit city 1,3,2,4,5
- Fitness: Total tour length (minimize)
- Selection: Tournament or roulette wheel
- Crossover: Order Crossover (OX) to preserve city ordering
- Mutation: Swap Mutation - randomly swap two cities
- Repeat until maximum generations or satisfactory tour length found

10-Mark Questions

Q7. [10M] Explain the complete flow of Genetic Algorithm with all steps and operators.

COMPLETE GA FLOW:

- Step 1 - INITIALIZATION: Create random population of N chromosomes. Each chromosome represents a potential solution.
- Step 2 - EVALUATION: Calculate fitness of each individual using fitness function.
- Step 3 - SELECTION: Select parents based on fitness. Fitter individuals have higher chance of selection.
- Step 4 - CROSSOVER: With probability pc, apply crossover to selected parents to create offspring.
- Step 5 - MUTATION: With probability pm, randomly alter genes of offspring.
- Step 6 - REPLACEMENT: Replace old population with new offspring.
- Step 7 - TERMINATION CHECK: If stopping criteria met, output best solution. Else go to Step 2.

CONSTRAINTS IN GA:

- Penalty Functions: Add penalty to fitness for constraint violation
- Repair Operators: Fix infeasible solutions after crossover/mutation
- Decoder: Decode chromosome to always produce feasible solution

GA VARIANTS:

- Canonical GA (Holland Classifier): Original simple GA with binary encoding

- Messy GA: Variable-length chromosomes, allows gene reordering
- Steady-State GA: Replace only worst individuals each generation
- Island Model: Multiple populations evolve independently with migration

Unit V: Computational Intelligence and NLP

2-Mark Questions

Q1. [2M] What is Word Embedding? Name common techniques.

Word Embedding is a technique to represent words as dense vectors of real numbers. Words with similar meanings have similar vector representations.

- Bag of Words (BoW): Counts word occurrences, no order
- TF-IDF: Term Frequency-Inverse Document Frequency
- Word2Vec: Neural network based, captures semantic relationships
- GloVe: Global Vectors, uses co-occurrence statistics

Q2. [2M] What is the BLEU Score?

BLEU (Bilingual Evaluation Understudy) Score is a metric for evaluating machine translation quality. It compares machine translation output with reference human translations. Score ranges 0 to 1 (or 0 to 100%). Higher score = better translation quality. Measures n-gram precision.

Q3. [2M] What is the BERT model?

BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained NLP model by Google. It reads text bidirectionally (both left-to-right and right-to-left) to understand context. Pre-trained on large corpus then fine-tuned for specific tasks: classification, QA, NER.

Q4. [2M] Differentiate TF-IDF and Word2Vec.

- TF-IDF: Sparse vector, no semantic meaning, easy to compute, not context-aware
- Word2Vec: Dense vector, captures semantics, requires training, context-aware
- TF-IDF: king and queen are unrelated; Word2Vec: similar vectors

5-Mark Questions

Q5. [5M] Explain Seq2Seq model and Neural Machine Translation.

Seq2Seq (Sequence to Sequence) is a neural network architecture for NLP tasks.

- Architecture: Encoder + Decoder
- Encoder: Reads input sequence and creates context vector (hidden state)
- Decoder: Generates output sequence from context vector
- Used for: Machine Translation, Text Summarization, Chatbots, QA

NEURAL MACHINE TRANSLATION:

- Traditional MT uses rules/statistics; NMT uses neural networks end-to-end
- Attention Mechanism: Allows decoder to focus on relevant input parts
- Better handles long sentences and rare words
- BLEU and BERT scores used to evaluate translation quality

Q6. [5M] Explain Word2Vec and GloVe word embedding techniques.

WORD2VEC (Mikolov et al., 2013):

- Two architectures: CBOW (predicts word from context) and Skip-gram (predicts context from word)
- Captures semantic similarity: king - man + woman = queen
- Trained on local context window

GLOVE (Global Vectors, Pennington et al., 2014):

- Uses global word-word co-occurrence matrix
- Captures both local and global context
- Better at capturing word analogies and relationships
- Both produce dense vector representations of words

10-Mark Questions

Q7. [10M] Explain BERT model architecture and its applications in detail.

BERT - Bidirectional Encoder Representations from Transformers

ARCHITECTURE:

- Based on Transformer encoder (not decoder)
- Reads entire sentence bidirectionally (unlike GPT which is left-to-right)
- Multiple stacked transformer layers with self-attention mechanism
- BERT-Base: 12 layers, 768 hidden units, 12 attention heads, 110M parameters
- BERT-Large: 24 layers, 1024 hidden units, 16 attention heads, 340M parameters

PRE-TRAINING TASKS:

- Masked Language Model (MLM): 15% of words masked, model predicts them
- Next Sentence Prediction (NSP): Predict if sentence B follows sentence A

FINE-TUNING:

- Pre-trained BERT + task-specific layer on top
- Fine-tuned with small task-specific dataset

APPLICATIONS:

- Question Answering (QA): SQuAD dataset benchmark
- Text Classification: Sentiment analysis, spam detection
- Named Entity Recognition (NER): Identify people, places, organizations
- Text Summarization and Generation
- Patient Triage using ChatGPT (case study)
- Semantic Search and Information Retrieval

Unit VI: Artificial Immune Systems

2-Mark Questions

Q1. [2M] What is an Artificial Immune System (AIS)?

AIS is a computational system inspired by the biological immune system. It uses immune system mechanisms (antibody-antigen interactions, clonal selection, negative selection) to solve engineering problems like optimization, classification, and anomaly detection.

Q2. [2M] What is Clonal Selection Theory?

Clonal Selection Theory explains how the immune system mounts a specific response to a foreign antigen. B-cells that recognize an antigen are selected, clonally amplified (copied many times), and the clones undergo hypermutation. Clones with higher affinity to the antigen are selected to fight infection.

Q3. [2M] Name and briefly explain the AIS models.

- Clonal Selection Model (CLONALG): Based on clonal selection and hypermutation
- Negative Selection Model: Generates detectors that recognize non-self patterns
- Network Theory Model (aiNet): Based on immune network theory
- Danger Theory Model: Responds to danger signals not just non-self

- Dendritic Cell Model: Based on dendritic cells that process antigens

Q4. [2M] What is hypermutation in AIS?

Hypermutation is a high rate of mutation applied to clones of selected antibodies. Unlike regular mutation, hypermutation is inversely proportional to affinity - lower affinity clones mutate more to explore solution space. High affinity clones mutate less to exploit good solutions.

5-Mark Questions

Q5. [5M] Explain the Clonal Selection Algorithm (CSA) with steps.

- Step 1 - Initialize: Create random population of antibodies
- Step 2 - Calculate Affinity: Evaluate how well each antibody solves the problem
- Step 3 - Select: Choose best antibodies based on affinity
- Step 4 - Clone: Create copies of selected antibodies. Better = more clones
- Step 5 - Hypermutate: Mutate clones inversely proportional to affinity
- Step 6 - Select Next Gen: Choose best from original + mutated clones
- Step 7 - Optionally: Replace worst with random new antibodies
- Step 8 - Repeat until convergence

KEY PRINCIPLE: Better solutions get more clones, clones with higher affinity mutate less

Q6. [5M] Explain the AIRS algorithm for classification.

AIRS (Artificial Immune Recognition System) for classification:

- Training Phase: Randomly select N samples as initial memory cells (detectors)
- Apply hypermutation to memory cells to improve coverage
- Prediction Phase: For new sample, calculate distance to all memory cells
- Euclidean Distance = sqrt(sum of (xi-yi)^2)
- Assign label of closest memory cell (nearest neighbor classification)

ADVANTAGES over kNN: Reduced memory cells needed, learned prototypes, immune-inspired learning

Application: Structure damage classification, pattern recognition

10-Mark Questions

Q7. [10M] Explain all AIS models with their principles and applications.

1. CLONAL SELECTION MODEL (CLONALG):

- Based on clonal selection theory of B-cells
- Best matching antibodies cloned and mutated
- Used for: Optimization, pattern recognition, machine learning

2. NEGATIVE SELECTION ALGORITHM:

- Inspired by T-cell maturation in thymus
- Self tolerance: T-cells that match self are eliminated
- Detectors generated to recognize non-self (anomalies)
- Used for: Anomaly detection, computer security, fault detection

3. IMMUNE NETWORK THEORY (aiNet):

- Based on Jerne's network theory
- Antibodies stimulate and suppress each other
- Network reaches stable state representing data structure
- Used for: Clustering, data compression, multimodal optimization

4. DANGER THEORY MODEL:

- Proposes immune system responds to danger signals not just non-self
- Damage to cells generates danger signals
- Used for: Intrusion detection, anomaly detection

5. DENDRITIC CELL MODEL:

- Based on role of dendritic cells in immune system
- Processes multiple signals: antigens, danger signals, safe signals
- Used for: Classification, anomaly detection in complex environments

APPLICATIONS OF AIS:

- Computer security and intrusion detection
- Medical diagnosis and pattern recognition
- Robotics and fault tolerance
- Optimization problems (CLONALG)
- Structural damage classification

417531: Distributed Computing

Unit I: Introduction to Distributed Computing

2-Mark Questions

Q1. [2M] What is a Distributed System?

A Distributed System is a collection of independent computers that appears to users as a single coherent system. The computers communicate and coordinate their actions by passing messages. Key characteristics: concurrency, no global clock, independent failures.

Q2. [2M] What are the goals of Distributed Systems?

- Resource Sharing: Share hardware, software, and data
- Openness: Interoperability with other systems using standards
- Scalability: Handle increasing load by adding more resources
- Transparency: Hide distribution from users (location, migration, replication)
- Fault Tolerance: Continue operating despite component failures

Q3. [2M] What are the types of Distributed Systems?

- Distributed Computing Systems: Cluster computing, Grid computing, Cloud computing
- Distributed Information Systems: Transaction processing systems, ERP
- Distributed Pervasive Systems: Mobile systems, sensor networks, IoT

Q4. [2M] What is fault tolerance in distributed systems?

Fault tolerance is the ability of a distributed system to continue operating correctly even when some components fail. Achieved through: Redundancy (multiple copies), Replication (data on multiple nodes), Checkpointing (save state periodically), Recovery mechanisms (restart from last checkpoint).

Q5. [2M] What are use cases of integrating AI in distributed computing?

- Predictive Maintenance: AI detects equipment failures before they occur
- Fraud Detection: ML models detect fraudulent transactions in real-time
- Intelligent Transportation: Optimize traffic flow and routing
- Supply Chain Optimization: Predict demand, optimize inventory
- Healthcare Diagnostics: Distributed AI for medical image analysis
- NLP: Distributed processing of large text datasets

5-Mark Questions

Q6. [5M] Explain the characteristics and issues of Distributed Systems.

CHARACTERISTICS:

- Concurrency: Multiple processes run simultaneously
- No Global Clock: Hard to synchronize events across nodes
- Independent Failures: Any component can fail independently
- Heterogeneity: Different hardware, OS, programming languages

ISSUES:

- Data Storage: Where to store data, how to replicate
- Data Consistency: Keeping replicas synchronized
- Communication Overhead: Network latency and bandwidth limits
- Synchronization: Coordinating concurrent processes

- Fault Tolerance: Handling node and network failures
- Security: Protecting distributed resources from attacks

Q7. [5M] Explain the Distributed System Models.

- 1. Client-Server Model: Clients request services from servers. Simple but server can be bottleneck. Example: Web browsing.
- 2. Peer-to-Peer (P2P) Model: All nodes are equal, act as both client and server. No central authority. Example: BitTorrent.
- 3. Three-Tier Model: Presentation, Logic, Data layers separated. Scalable web applications.
- 4. Multi-Tier / N-Tier: Multiple layers of services. Enterprise applications.
- 5. Cloud Model: Services provided over internet. IaaS, PaaS, SaaS models.

10-Mark Questions

Q8. [10M] Explain the fundamentals of Distributed Computing with AI integration in detail.

FUNDAMENTALS OF DISTRIBUTED COMPUTING:

- Definition: Multiple computers working together as single system
- Communication: Message passing through network (TCP/IP, RPC, RMI)
- Coordination: Processes must coordinate to avoid conflicts
- Replication: Data copied to multiple nodes for availability
- Transparency: Users unaware of distribution

AI IN DISTRIBUTED COMPUTING:

- Distributing Computational Tasks: Large ML models split across GPU clusters
- Handling Large Volumes of Data: Distributed data processing pipelines
- Parallel Processing: Multiple nodes process data simultaneously

CHALLENGES OF AI IN DISTRIBUTED SYSTEMS:

- Data Storage: Distributed databases, HDFS, NoSQL stores
- Data Consistency: CAP theorem - Consistency, Availability, Partition tolerance
- Communication Overhead: Gradient sharing in distributed ML is expensive
- Synchronization: Ensuring all nodes use consistent model parameters
- Fault Tolerance: Training continues even if some nodes fail

USE CASES:

- Fraud Detection: Banks use distributed ML to detect fraud across millions of transactions
- Healthcare: Distributed AI for MRI analysis, patient monitoring
- Supply Chain: Distributed optimization for inventory management
- NLP: Processing millions of documents in parallel

Unit II: Distributed Data Management and Storage

2-Mark Questions

Q1. [2M] What is HDFS?

HDFS (Hadoop Distributed File System) is a distributed file system designed to store very large files across multiple machines. Key features: Master-Slave architecture (NameNode + DataNodes), fault tolerant (default 3x replication), high throughput access, designed for batch processing.

Q2. [2M] What is the difference between Strong Consistency and Eventual Consistency?

- Strong Consistency: After write completes, all reads return the new value. All nodes see same data simultaneously. Lower availability.

• Eventual Consistency: Reads may return stale data temporarily, but eventually all nodes converge to same value. Higher availability. Used in Amazon DynamoDB, DNS.

Q3. [2M] What are Distributed Hash Tables (DHTs)?

DHTs are distributed systems that provide a lookup service similar to hash tables. Key-value pairs are stored across multiple nodes. Any node can retrieve a value by key efficiently. Used in P2P systems. Examples: Chord, Kademlia, Pastry. Lookup is O(log n) hops.

Q4. [2M] What is Edge Computing?

Edge Computing is a paradigm that brings computation and data storage closer to the source of data (edge of network), rather than relying on centralized cloud servers. Benefits: reduced latency, bandwidth savings, real-time processing, works with poor connectivity. Used in IoT, autonomous vehicles, AR/VR.

Q5. [2M] What are Message Brokers and Stream Processing?

• Message Brokers: Middleware that enables communication between distributed systems by routing, buffering, and delivering messages. Examples: Apache Kafka, RabbitMQ, ActiveMQ.

• Stream Processing: Real-time processing of continuous data streams. Examples: Apache Kafka Streams, Apache Flink, Apache Storm.

5-Mark Questions

Q6. [5M] Explain the Data Replication models in Distributed Systems.

• 1. Eager Replication: All replicas updated synchronously before commit. Strong consistency but high latency.

• 2. Lazy Replication: Primary updated first, others updated asynchronously. Better performance, eventual consistency.

• 3. Quorum-Based: Write must succeed on majority of nodes, read from majority. Balance between consistency and availability.

• 4. Consensus-Based: Paxos/Raft algorithm ensures all nodes agree. Used in critical systems.

• 5. Selective Replication: Only specific data replicated based on access patterns.

Q7. [5M] Explain GFS and HDFS architecture.

GFS (Google File System):

• Master server stores metadata, multiple ChunkServers store data

• Files divided into 64MB chunks, each chunk replicated 3 times

• Optimized for large sequential reads, not random access

HDFS (Hadoop Distributed File System):

• NameNode: Master node, stores file system metadata and block locations

• DataNodes: Store actual data blocks (default 128MB blocks)

• Secondary NameNode: Periodically merges NameNode edit logs

• Replication Factor: Default 3 copies per block

• Rack Awareness: Replicas placed on different racks for fault tolerance

10-Mark Questions

Q8. [10M] Explain all Distributed Computing Frameworks and Technologies in detail.

1. PARALLEL COMPUTING:

• Multiple processors work on same problem simultaneously

• Shared memory (OpenMP) or distributed memory (MPI)

2. DISTRIBUTED COMPUTING MODELS:

• Message Passing: Processes communicate by sending/receiving messages

- Shared Memory: All processes access same memory space

3. HDFS:
   - Block-based storage, default block size 128MB
   - NameNode + DataNode architecture
   - Write-once-read-many model for big data
4. CLOUD COMPUTING PLATFORMS:
   - AWS (Amazon): EC2, S3, EMR for Hadoop, Lambda for serverless
   - Azure (Microsoft): HDInsight, Blob Storage, Azure ML
   - GCP (Google): BigQuery, Dataproc, Cloud Storage
5. MESSAGE BROKERS:
   - Apache Kafka: High-throughput distributed streaming platform
   - RabbitMQ: Advanced Message Queuing Protocol (AMQP)
6. EDGE COMPUTING:
   - Compute at network edge, reduces cloud dependency
   - Critical for IoT, real-time applications, autonomous vehicles
7. CLUSTER COMPUTING:
   - Multiple nodes connected via high-speed network
   - Job schedulers: YARN, Mesos, Kubernetes

Unit III: Distributed Computing Algorithms

2-Mark Questions

Q1. [2M] What is a Distributed Consensus Algorithm?

A distributed consensus algorithm enables all nodes in a distributed system to agree on a single value even in the presence of failures. Examples: Paxos, Raft, ZAB, Viewstamped Replication. Used in leader election, distributed transactions, and replicated state machines.

Q2. [2M] What is the RAFT algorithm?

RAFT is a consensus algorithm designed to be more understandable than Paxos. It has three server states: Leader, Follower, Candidate. Leader handles all client requests, replicates log to followers. If leader fails, new election is held. Used in: etcd, CockroachDB, TiKV.

Q3. [2M] What are Load Balancing strategies?

- Round Robin: Distribute requests equally in rotation
- Least Connection: Send to server with fewest active connections
- Weighted Round Robin: Servers with higher capacity get more requests
- Random: Randomly select server
- Dynamic: Based on real-time server load metrics
- Predictive: AI predicts future load to proactively balance

Q4. [2M] What is fault tolerance in distributed systems?

- Redundancy: Multiple copies of components
- Replication: Same data on multiple nodes
- Checkpointing: Save state periodically for recovery
- Heartbeat: Monitor node health, detect failures
- Failover: Automatic switch to backup when primary fails

5-Mark Questions

Q5. [5M] Explain how AI techniques optimize distributed computing algorithms.

- 1. ML for Resource Allocation: ML predicts resource needs, allocates CPU/memory proactively
- 2. Reinforcement Learning for Load Balancing: RL agent learns optimal load distribution policy through trial and error
- 3. Genetic Algorithms for Task Scheduling: GA finds optimal assignment of tasks to processors minimizing makespan
- 4. Swarm Intelligence for Optimization: ACO and PSO solve distributed routing and scheduling problems
- 5. Neural Networks for Anomaly Detection: Detect and predict node failures before they occur

Q6. [5M] Explain Paxos consensus algorithm.

Paxos is a consensus protocol for distributed systems with 3 roles:

- Proposer: Proposes values to be agreed upon
- Acceptor: Accepts or rejects proposals
- Learner: Learns the final agreed value

TWO PHASES:

- Phase 1 (Prepare): Proposer sends Prepare(n) to majority of acceptors. Acceptors promise not to accept proposals with number < n
- Phase 2 (Accept): Proposer sends Accept(n, value). If majority accept, consensus reached

Variants: Fast Paxos, Multi-Paxos, Egalitarian Paxos

Used in: Google Chubby, Apache Zookeeper (ZAB variant)

10-Mark Questions

Q7. [10M] Explain Communication and Coordination in Distributed Systems with all algorithms.

COMMUNICATION IN DISTRIBUTED SYSTEMS:

- Remote Procedure Call (RPC): Call procedures on remote machines as if local
- Remote Method Invocation (RMI): Java-based RPC with object orientation
- Message Passing Interface (MPI): Standard for parallel computing communication
- REST APIs: HTTP-based communication for web services
- gRPC: Google's high-performance RPC framework

COORDINATION ALGORITHMS:

- Mutual Exclusion: Only one process accesses critical section at a time
- - Centralized: Coordinator grants permission (single point of failure)
- - Ring-based: Token passed in ring, holder enters CS
- - Distributed: Ricart-Agrawala algorithm using timestamps

CONSENSUS ALGORITHMS:

- Paxos: Two-phase protocol, handles failures, complex to implement
- Raft: Leader-based, more understandable than Paxos
- ZAB (Zookeeper Atomic Broadcast): Used by Apache Zookeeper
- PBFT: Byzantine fault tolerant consensus

CLOCK SYNCHRONIZATION:

- Cristian's Algorithm: Client requests time from server, accounts for RTT
- NTP (Network Time Protocol): Hierarchical time synchronization
- Lamport Clocks: Logical clocks for event ordering
- Vector Clocks: Capture causal relationships between events

Unit IV: Distributed Machine Learning and AI

2-Mark Questions

Q1. [2M] What is Federated Learning?

Federated Learning is a distributed ML approach where model is trained across multiple devices/nodes without sharing raw data. Each node trains on local data, sends only model updates (gradients) to central server. Preserves data privacy. Used by Google (keyboard predictions), healthcare (patient data).

Q2. [2M] What is Data Parallelism vs Model Parallelism?

- Data Parallelism: Same model copied to multiple nodes, each node trains on different data subset. Gradients aggregated. Most common approach.
- Model Parallelism: Large model split across multiple nodes, each node has different layers. Used when model too large for single GPU.

Q3. [2M] What is Apache Spark?

Apache Spark is an open-source distributed computing framework for big data processing. Key features: In-memory computing (100x faster than Hadoop MapReduce), supports batch and streaming, ML library (MLlib), SQL, graph processing. Uses RDD (Resilient Distributed Dataset) as core abstraction.

Q4. [2M] What is Distributed Gradient Descent?

Distributed Gradient Descent is gradient descent optimization performed across multiple machines. Types: Synchronous (all nodes must complete before aggregation), Asynchronous (nodes update independently). Algorithms: All-Reduce, Parameter Server, Hogwild!

5-Mark Questions

Q5. [5M] Explain Federated Learning with its advantages and challenges.

FEDERATED LEARNING PROCESS:

- 1. Central server sends global model to selected clients
- 2. Each client trains model on local data
- 3. Clients send model updates (gradients/weights) to server
- 4. Server aggregates updates (FedAvg algorithm)
- 5. Updated global model sent to clients again

ADVANTAGES:

- Privacy: Raw data never leaves device
- Reduces communication: Only gradients sent, not data
- Personalization: Models can be personalized per client

CHALLENGES:

- Non-IID data: Data distribution differs across clients
- Communication bottleneck: Gradient aggregation overhead
- Stragglers: Slow clients delay synchronous training

Q6. [5M] Explain the software tools for Distributed Machine Learning.

- Apache Spark MLlib: Scalable ML library on Spark. Supports classification, regression, clustering, collaborative filtering.
- TensorFlow: Google's ML framework with distributed training support using TF.distribute strategies.
- PyTorch Distributed: Data parallel, model parallel, RPC-based distributed training.
- GraphLab: Graph-based distributed ML framework for collaborative filtering, graph analytics.
- Petuum: Parallel ML system with Bösen parameter server and Strads scheduler.
- Horovod: Uber's framework for distributed deep learning using all-reduce.

10-Mark Questions

Q7. [10M] Explain Distributed Machine Learning architectures and algorithms in detail.

DISTRIBUTED ML ARCHITECTURES:

1. Parameter Server Architecture:

- Workers: Compute gradients using local data
- Parameter Server: Aggregates gradients, updates global model
- Workers pull updated parameters, push gradients
- Synchronous vs Asynchronous updates

2. All-Reduce Architecture:

- All nodes communicate with each other directly
- Ring-AllReduce: Efficient bandwidth utilization
- Used by Horovod, NCCL (NVIDIA)

DISTRIBUTED ML ALGORITHMS:

1. Data Parallelism with SGD:

- Each worker has copy of model, different data batch
- Gradients averaged across all workers
- Synchronous: Wait for all workers (consistent but slow)
- Asynchronous: No waiting (fast but inconsistent, Hogwild!)

2. Model Parallelism:

- Model split across devices by layers
- Pipeline parallelism: stages of model on different GPUs
- Used for GPT, BERT training with billions of parameters

3. Federated Learning:

- FedAvg: Weighted average of local model updates
- FedProx: Adds proximal term to handle heterogeneous data

4. Elastic Averaging SGD (EASGD):

- Workers maintain local model + elastic force towards center
- Better exploration of parameter space

Unit V: Big Data Processing in Distributed Systems

2-Mark Questions

Q1. [2M] What is Hadoop MapReduce?

Hadoop MapReduce is a programming model for processing large datasets in parallel. Two phases: Map (process input, emit key-value pairs) and Reduce (aggregate values by key). Runs on HDFS. Fault tolerant, scalable, but high latency due to disk I/O.

Q2. [2M] What is Apache Spark and how is it better than Hadoop?

- Spark is faster than Hadoop MapReduce because it uses in-memory processing (DAG vs MR stages)
- Spark: 100x faster for in-memory, 10x for on-disk processing
- Spark supports: Batch, Streaming, ML, SQL, Graph in unified framework
- Hadoop MapReduce: Disk-based, only batch, no iterative ML support

Q3. [2M] What is SIMD and MIMD in parallel processing?

- SISD: Single Instruction Single Data - traditional sequential processor

- MISD: Multiple Instruction Single Data - rare, fault-tolerant systems
- SIMD: Single Instruction Multiple Data - GPU processing (same op on many data)
- MIMD: Multiple Instruction Multiple Data - most distributed systems
- SPMD: Single Program Multiple Data - MPI-based parallel computing

Q4. [2M] What is the difference between Real-time Analytics and Streaming Analytics?

- Real-time Analytics: Process and analyze data immediately as it arrives. Millisecond latency. Examples: fraud detection, stock trading.
- Streaming Analytics: Continuous processing of data streams over time windows. Can tolerate slight delay. Examples: log analysis, sensor data aggregation.

5-Mark Questions

Q5. [5M] Explain Big Data Processing frameworks in Distributed Computing.

- 1. Hadoop MapReduce: Batch processing, fault tolerant, disk-based, good for simple transformations
- 2. Apache Spark: In-memory processing, unified batch+streaming+ML, 100x faster than Hadoop
- 3. Apache Storm: Real-time stream processing, low latency, processes millions of tuples/second
- 4. Apache Samza: Stream processing built on Kafka, stateful processing, LinkedIn's choice
- 5. Apache Flink: Unified batch and stream processing, true streaming (not micro-batches), event time processing

Q6. [5M] Explain Scalable Data Ingestion in Distributed Systems.

Data Ingestion is the process of obtaining, importing, and processing data for storage and analysis.

TYPES OF INGESTION:

- Batch Ingestion: Collect data over time and process in batches. Simple but high latency.
- Real-time/Streaming Ingestion: Process data as soon as it arrives. Low latency.
- Micro-batch: Small batches processed frequently (Spark Streaming)

TOOLS:

- Apache Kafka: Distributed streaming platform for ingesting high-volume data
- Apache Flume: Collecting log data from distributed sources
- Apache Sqoop: Transfer data between Hadoop and relational databases
- AWS Kinesis: Managed streaming data service

CHALLENGES:

- Schema evolution, data quality, exactly-once semantics, backpressure handling

10-Mark Questions

Q7. [10M] Explain MapReduce with word count example and its phases in detail.

MAPREDUCE PROGRAMMING MODEL:

- Proposed by Google (Dean & Ghemawat, 2004)
- Handles parallelization, fault tolerance, data distribution automatically

5 PHASES:

- 1. Input Splits: Input divided into fixed-size chunks (default 128MB)
- 2. Map Phase: Each mapper processes one split, emits (key, value) pairs
- 3. Shuffle and Sort: Framework groups all values by key, sorts keys
- 4. Reduce Phase: Reducer processes all values for each key, emits output
- 5. Output: Written to HDFS

WORD COUNT EXAMPLE:

- Input: 'This is an apple. Apple is red in color.'
- Map Output: (this,1),(is,1),(an,1),(apple,1),(apple,1),(is,1),(red,1)...
- After Sort: (apple,[1,1]),(is,[1,1]),(this,[1])...
- Reduce Output: (apple,2),(is,2),(this,1),(an,1),(red,1),(in,1),(color,1)

HADOOP STREAMING (Python MapReduce):

- CharMapper.py: Reads lines, emits each char with count 1
- CharReducer.py: Aggregates counts for same character
- Command: cat input.txt | python3 CharMapper.py | sort | python3 CharReducer.py

LIMITATIONS:

- Disk I/O intensive - writes intermediate data to disk
- Not suitable for real-time or iterative algorithms
- High setup overhead for small datasets

Unit VI: Distributed Systems Security and Privacy

2-Mark Questions

Q1. [2M] What are the security challenges in Distributed Systems?

- Authentication: Verifying identity of nodes and users
- Authorization: Controlling access to resources
- Data Integrity: Ensuring data not tampered in transit
- Confidentiality: Preventing unauthorized data access
- Availability: Ensuring system accessible despite attacks
- Insider Threats: Malicious actions by authorized users

Q2. [2M] What is TLS/SSL?

TLS (Transport Layer Security) / SSL (Secure Sockets Layer) is a cryptographic protocol for secure communication over networks. Provides: Encryption (data confidentiality), Authentication (verify identity using certificates), Integrity (detect tampering using MACs). Used in HTTPS, email, VPN. TLS 1.3 is current standard.

Q3. [2M] What is Differential Privacy?

Differential Privacy is a mathematical framework for privacy-preserving data analysis. It ensures that the output of a computation is nearly the same whether any individual's data is included or not. Achieved by adding carefully calibrated random noise. Used by Apple, Google, US Census Bureau.

Q4. [2M] What is Federated Learning's role in privacy?

Federated Learning preserves privacy by training ML models on decentralized data. Raw data never leaves the device. Only model updates (gradients) are shared. Combined with Differential Privacy and Secure Aggregation, it provides strong privacy guarantees. Used in healthcare, mobile keyboards, financial fraud detection.

Q5. [2M] What is Homomorphic Encryption?

Homomorphic Encryption allows computations to be performed on encrypted data without decrypting it. The result, when decrypted, is the same as if operations were performed on plaintext. Types: Partially, Somewhat, Fully Homomorphic Encryption. Enables secure cloud computing and privacy-preserving ML.

5-Mark Questions

Q6. [5M] Explain AI-based Intrusion Detection Systems.

AI-based IDS use ML/DL to detect malicious activities in distributed systems:

• 1. Anomaly Detection: Learn normal behavior baseline, flag deviations. ML models: autoencoders, isolation forests, one-class SVM.

• 2. Behavior-based Detection: Analyze patterns of user/entity behavior (UEBA - User and Entity Behavior Analytics).

• 3. Threat Intelligence: ML analyzes threat feeds, vulnerability databases to predict attacks.

• 4. Real-time Response: Automatically block suspicious IPs, quarantine affected nodes.

• 5. Adaptive Security: ML models continuously retrain on new attack patterns.

ADVANTAGES over traditional IDS:

• Detects zero-day attacks and unknown threats

• Reduces false positives through learning

• Scales to large distributed environments

Q7. [5M] Explain Privacy Preservation Techniques in Distributed Systems.

• 1. Differential Privacy: Add mathematical noise to queries/outputs. Guarantees individual records cannot be identified.

• 2. Homomorphic Encryption: Compute on encrypted data. Cloud can process without seeing actual data.

• 3. Secure Multi-Party Computation (SMPC): Multiple parties jointly compute function without revealing inputs. Example: Private salary comparison.

• 4. Federated Learning: Train ML models without centralizing data.

• 5. Anonymization: Remove direct identifiers (name, ID). k-anonymity ensures each record indistinguishable from k-1 others.

• 6. Pseudonymization: Replace identifiers with pseudonyms. Reversible with key. GDPR compliant.

10-Mark Questions

Q8. [10M] Explain Security in Distributed Systems covering all major aspects.

SECURITY CHALLENGES:

• Distributed systems have larger attack surface than centralized systems

• Network attacks, node compromises, data breaches, DoS attacks

ENCRYPTION AND SECURE COMMUNICATION:

• TLS/SSL: Encrypts data in transit using asymmetric (handshake) + symmetric (data) encryption

• PKI (Public Key Infrastructure): Certificate authority issues digital certificates

• VPN: Creates encrypted tunnel over public network

• AMQP with TLS: Secure message broker communication

ACCESS CONTROL:

• RBAC (Role-Based): Access based on user roles

• ABAC (Attribute-Based): Access based on attributes of user, resource, environment

• OAuth 2.0 / JWT: Token-based authorization for APIs

PRIVACY TECHNIQUES:

• Differential Privacy: epsilon-differential privacy, add Laplace/Gaussian noise

• Homomorphic Encryption: FHE allows arbitrary computation on encrypted data

• SMPC: Secret sharing, garbled circuits, oblivious transfer protocols

• Federated Learning: Gradient compression, secure aggregation, DP-FL

AI FOR SECURITY:

• UEBA: Baseline normal behavior, detect deviations

• Threat Hunting: Proactively search for threats using ML

• Automated Response: SOAR (Security Orchestration, Automation and Response)

- DeepFake Detection: Using AI to detect AI-generated threats

CASE STUDY - Healthcare:

- HIPAA compliance requires: encryption at rest and in transit, access logging, audit trails
- Federated learning enables multi-hospital ML without sharing patient data

IMPORTANT SHORT NOTES FOR EXAM

CI - Key Definitions to Remember

Fuzzy Set: Set where each element has membership degree between 0 and 1. More flexible than crisp sets.

Membership Function: Function that maps elements to membership values [0,1]. Types: Triangular, Trapezoidal, Gaussian.

Fuzzification: Converting crisp input to fuzzy membership values.

Defuzzification: Converting fuzzy output to crisp value. Methods: Centroid, MOM.

Genetic Algorithm: Evolutionary optimization inspired by Darwin's natural selection using selection, crossover, mutation.

Fitness Function: Measures quality of a solution in GA. Better fitness = better solution.

Crossover: Combining two parent chromosomes to create offspring in GA.

Mutation: Random change in chromosome to maintain diversity in GA population.

Clonal Selection: AIS mechanism where best antibodies are cloned and mutated. Better = more clones.

Affinity: Measure of how well an antibody matches an antigen. Higher affinity = better solution.

BERT: Bidirectional Encoder Representations from Transformers. Pre-trained NLP model by Google.

Word2Vec: Neural network technique to represent words as dense vectors capturing semantic meaning.

TF-IDF: Term Frequency-Inverse Document Frequency. Statistical measure of word importance in documents.

ACO: Ant Colony Optimization. Swarm intelligence inspired by ant foraging behavior. Uses pheromone trails.

BLEU Score: Bilingual Evaluation Understudy. Metric for evaluating machine translation quality.

DC - Key Definitions to Remember

Distributed System: Collection of independent computers appearing as single coherent system to users.

Fault Tolerance: Ability to continue operating correctly despite component failures using redundancy.

HDFS: Hadoop Distributed File System. Stores large files across multiple machines with 3x replication.

MapReduce: Programming model with Map (emit key-value) and Reduce (aggregate) phases for big data.

Consensus Algorithm: Algorithm to make all nodes agree on single value despite failures. Examples: Paxos, Raft.

Federated Learning: Distributed ML where model trained across devices without sharing raw data.

Data Parallelism: Same model on multiple nodes, each processes different data. Gradients averaged.

Model Parallelism: Large model split across multiple devices by layers.

Differential Privacy: Mathematical framework adding noise to protect individual privacy in data analysis.

Homomorphic Encryption: Compute on encrypted data without decrypting. Result same as plaintext computation.

Eventual Consistency: All nodes eventually converge to same value, may be temporarily inconsistent.

Strong Consistency: All reads return most recent write. Higher consistency, lower availability.

CAP Theorem: Distributed system can only guarantee 2 of 3: Consistency, Availability, Partition Tolerance.

Load Balancing: Distributing workload across multiple servers to optimize resource use.

Edge Computing: Processing data at network edge, close to data source. Reduces latency.

Apache Spark: In-memory distributed computing. 100x faster than Hadoop for iterative algorithms.

RAFT: Consensus algorithm more understandable than Paxos. Uses leader election and log replication.

TLS/SSL: Cryptographic protocol for secure communication. Provides encryption, authentication, integrity.

Most Important / Likely Exam Questions

No.	Question	Subject	Marks
1	Explain Fuzzy Set operations: Union, Intersection, Complement, Difference with examples	**CI**	10
2	Explain Genetic Algorithm complete flow with all operators	**CI**	10
3	Explain Clonal Selection Algorithm with steps and applications	**CI**	10
4	Explain BERT model architecture and applications in NLP	**CI**	10
5	Explain Evolutionary Computing types and Ant Colony Optimization	**CI**	10
6	Explain all AIS models (Clonal Selection, Negative Selection, Network Theory, Danger Theory)	**CI**	10
7	What is Max-Min Composition? Explain with example	**CI**	5
8	Explain Seq2Seq model and Neural Machine Translation	**CI**	5
9	Explain Distributed System characteristics, issues and goals	**DC**	10
10	Explain MapReduce with all phases and Word Count example	**DC**	10
11	Explain Federated Learning with advantages and challenges	**DC**	10
12	Explain Security in Distributed Systems (TLS, Differential Privacy, SMPC, Homomorphic)	**DC**	10
13	Explain Big Data Processing frameworks (Hadoop, Spark, Storm, Flink)	**DC**	10
14	Explain Distributed ML architectures (Parameter Server, All-Reduce)	**DC**	10
15	Explain HDFS architecture and GFS	**DC**	5
16	Explain Consensus Algorithms: Paxos and Raft	**DC**	5
17	Explain CAP Theorem with examples	**DC**	5
18	Explain Load Balancing strategies in distributed systems	**DC**	5
19	What are linguistic variables and hedges in fuzzy logic?	**CI**	2
20	Differentiate AI and Computational Intelligence	**CI**	2

--- END OF QUESTION BANK ---